

# Sparse-View CT Reconstruction on the Mayo Dataset

Variational, End-to-End, and Generative Approaches

Group B – Computational Imaging Project

May 25, 2026

## Abstract

This report studies sparse-view computed tomography (CT) reconstruction as a computational imaging inverse problem. The same degraded Mayo CT measurements are used throughout: images resized to  $256 \times 256$ , parallel-beam projection geometries with 180, 90, 60, and 45 views, and additive Gaussian measurement noise with relative level 0.005. Three methodological families are compared: Total  $p$ -Variation (TpV) regularization—a classical model-based approach that encodes an explicit sparsity prior on image gradients; a Generalized ResUNet post-processing network—a supervised end-to-end method trained from filtered back-projection (FBP) proxy images; and a DiffPIR-based diffusion reconstruction method—a plug-and-play generative prior that guides a learned reverse diffusion process with CT data-consistency updates. Quantitative evaluation is performed through PSNR and SSIM, complemented by visual comparison panels across the four sparse-view configurations.

## 1 Introduction

Computational imaging is the joint design of image acquisition, the physical forward model, and computational reconstruction algorithms in order to recover information that a direct observation cannot provide [1]. Computed tomography is one of its most prominent instances: the internal attenuation map of an object is estimated from a collection of indirect X-ray line-integral measurements.

**Forward model.** In the two-dimensional setting of this project, the unknown image is  $x \in \mathbb{R}^{256 \times 256}$  and the acquisition follows the discrete linear forward model

$$y^\delta = K_\theta x + e, \quad (1)$$

where  $K_\theta : \mathbb{R}^N \rightarrow \mathbb{R}^M$  is the parallel-beam Radon-type projection operator parameterized by the set of projection angles  $\theta$ ,  $y^\delta \in \mathbb{R}^M$  is the noisy measured sinogram, and  $e \in \mathbb{R}^M$  is additive measurement noise. The operator  $K_\theta$  is a large, severely ill-conditioned matrix: its singular values decay rapidly to zero, so small perturbations in the data  $y^\delta$  produce arbitrarily large errors in any naive inversion. This property is the mathematical signature of an *ill-posed* inverse problem in the sense of Hadamard—existence, uniqueness, or stability of the solution may fail.

**Sparse-view task.** The project addresses the sparse-view CT scenario, in which the number of projection angles is strongly reduced relative to the Nyquist-Shannon angular sampling rate required for exact inversion:

$$n_\theta \in \{180, 90, 60, 45\}. \quad (2)$$

Table 1: Mayo processed dataset organization used by the project notebooks.

| Split      | Patients | Slices |
|------------|----------|--------|
| Train      | 8        | 2585   |
| Validation | 2        | 721    |
| Test       | 1        | 327    |

As  $n_\theta$  decreases, the operator  $K_\theta$  loses rows, reducing the measured data and increasing the degree of under-determination. In practice, a direct filtered back-projection (FBP) reconstruction from sparse-view data exhibits characteristic streak artifacts that grow more severe as  $n_\theta$  decreases. Regularization—either explicit through a variational penalty or implicit through a learned prior—is therefore indispensable.

**Methodological families.** Three reconstruction families are studied, each corresponding to one of the principal strategies introduced in the course:

1. **Model-based variational approach:** Total  $p$ -Variation (TpV) regularization [2], which expresses prior knowledge as an explicit penalty on the image gradient magnitude.
2. **End-to-end supervised learning:** a Generalized ResUNet post-processor [3] that maps an FBP proxy directly to a clean reconstruction by minimizing a supervised loss over training data.
3. **Diffusion-based generative prior:** DiffPIR [4], which embeds the CT measurement constraint inside the reverse denoising trajectory of a diffusion model.

## Mayo Dataset

The dataset used is the Mayo CT dataset provided in the course material [1]. PNG slices are organized by patient folders and split at patient level to avoid data leakage between train and test sets. The processed split used by the project notebooks is summarized in Table 1. Each slice is resized to  $256 \times 256$  using bilinear interpolation and normalized to  $[0, 1]$  before the CT forward model is applied. Processed data are saved as one `.pt` file per patient, preserving the correspondence between patient identifiers and slices.

For each clean image  $x$ , four degraded sinograms are generated, one per view configuration. The noise model is applied in the measurement domain: if  $y = K_\theta x$  is the clean sinogram, the noisy data are constructed as

$$y^\delta = y + e, \quad \frac{\|e\|_2}{\|y\|_2} \approx 0.005, \quad (3)$$

where  $e$  is sampled from a Gaussian distribution and scaled to match the prescribed relative level. Fixing the same noise realization for all methods guarantees a fair comparison: different noise instances would make numerical results non-comparable.

## Tools

The implementation uses the IPPy library distributed with the course material. The main components employed are:

- `IPPy.operators.CTProjector`: wraps the ASTRA-based parallel-beam forward operator  $K_\theta$  and its adjoint  $K_\theta^\top$  [5];

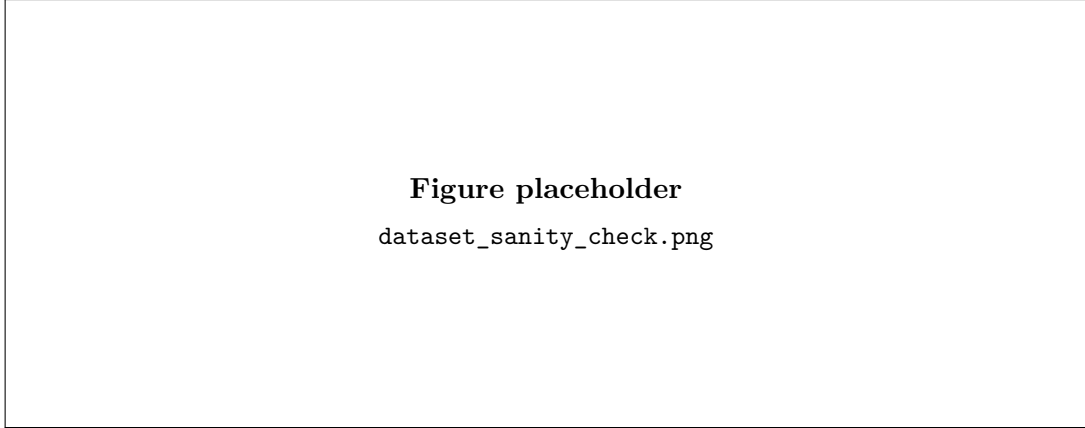


Figure 1: One normalized Mayo CT slice and the corresponding noisy sparse-view sinograms for 180, 90, 60, and 45 projection angles.

- `IPPy.solvers.FBP`: computes the Ram-Lak filtered back-projection proxy  $x_{\text{FBP}}$ , used as input to the ResUNet and as a baseline for comparison;
- `IPPy.solvers.ChambollePockTpVUnconstrained`: implements the primal-dual TpV solver via Chambolle–Pock iterations [6];
- `IPPy.nn.models.UNet`: provides the ResUNet architecture used for end-to-end post-processing [3];
- `IPPy.utilities.metrics`: computes PSNR and SSIM [7].

## 2 Selected Methods

### 2.1 Total $p$ -Variation Regularization

**Variational formulation.** The first selected method is a model-based variational approach. Given the ill-posed nature of the CT inverse problem, a principled remedy is to replace the naive inversion by a regularized variational problem that balances data fidelity with a prior on the image. The general *Tikhonov-type* formulation is

$$\hat{x} = \arg \min_{x \geq 0} \frac{1}{2} \|K_{\theta} x - y^{\delta}\|_2^2 + \lambda R(x), \quad (4)$$

where the first term enforces consistency with the measured sinogram and the second introduces prior information on the solution. The regularization parameter  $\lambda > 0$  controls the trade-off: small  $\lambda$  favors data-fidelity and may leave noise and streak artifacts; large  $\lambda$  enforces strong regularity but risks over-smoothing and loss of fine anatomical detail.

**Total  $p$ -Variation prior.** In this project, the regularizer is the Total  $p$ -Variation functional [2]:

$$R_p(x) = \sum_i \|\nabla x_i\|_2^p, \quad 0.1 < p < 0.5. \quad (5)$$

Here  $\nabla x_i \in \mathbb{R}^2$  is the discrete image gradient at pixel  $i$ , computed via finite differences. The standard total variation (TV) corresponds to  $p = 1$ , which yields a convex functional and is well known to promote piecewise-constant images while preserving sharp edges. By setting  $p < 1$ , the TpV penalty is *non-convex* and promotes *sparser* image gradients than TV: only a few pixels are allowed to have large gradient magnitudes, while the majority are forced to zero. This

property aligns well with the structure of CT anatomical images, which typically exhibit smooth regions separated by clinically meaningful edges (organ boundaries, bone surfaces). The project specifications require  $p \in (0.1, 0.5)$ ; we use  $p = 0.35$ .

**Chambolle–Pock primal-dual solver.** Because  $R_p$  is non-convex for  $p < 1$ , the optimization problem (4)–(5) cannot be solved by standard convex proximal algorithms. IPPy handles this through a *majorization-minimization* (reweighted) strategy: the TpV term is linearized around the current estimate by introducing pixel-wise weights

$$W^{(k)}(x) = \left( \frac{\sqrt{\eta^2 + |\nabla x^{(k)}|^2}}{\eta} \right)^{p-1}, \quad (6)$$

where  $\eta > 0$  is a small smoothing parameter that avoids singularities at zero-gradient locations. At each outer iteration  $k$ , the weights  $W^{(k)}$  are fixed and a weighted TV subproblem is solved via the *Chambolle–Pock primal-dual* scheme [6]. The algorithm alternates between a gradient-ascent step in the dual variable (associated with the TV-like term) and a gradient-descent step in the primal variable (associated with data fidelity), with step sizes governed by the operator norms of  $K_\theta$  and  $\nabla$ . Since  $p - 1 < 0$ , pixels with larger gradient magnitudes receive smaller weights in subsequent iterations, which progressively concentrates the sparsity pattern. The reconstruction is constrained to non-negative values at every step.

## 2.2 Generalized ResUNet Post-Processing

**FBP proxy and learned correction.** The second selected method is a supervised end-to-end approach. Rather than operating on the sinogram directly, the method first computes a filtered back-projection reconstruction from the sparse-view noisy data, then trains a neural network to map the FBP image to the clean slice. The two-stage pipeline is

$$x_{\text{FBP}} = \text{FBP}(y^\delta), \quad \hat{x} = f_\Theta(x_{\text{FBP}}), \quad (7)$$

where  $f_\Theta$  is the learned ResUNet and  $\Theta$  denotes its trainable parameters. The FBP step is not a standalone reconstruction method in this context: it is used only to convert the sinogram to the image domain, making the learning problem an image-to-image regression task rather than a sinogram-to-image problem.

**Supervised MSE loss.** The network is trained by minimizing the mean squared error between prediction and the corresponding clean Mayo slice:

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{j=1}^N \|f_\Theta(x_{\text{FBP},j}) - x_j\|_2^2. \quad (8)$$

Minimizing the  $\ell^2$  loss is equivalent, under a Gaussian likelihood model, to computing the maximum-likelihood estimate of the network output. In imaging, MSE training is known to produce outputs that are good in expectation but may appear slightly over-smoothed, because the network averages over residual uncertainty in the target distribution.

**ResUNet architecture.** The network architecture is the U-Net [3], extended with residual blocks in the encoder and decoder branches. The encoder extracts increasingly abstract features through successive downsampling blocks, while the decoder reconstructs a full-resolution output using transposed convolutions. Skip connections between encoder and decoder levels concatenate feature maps of the same spatial resolution, preserving fine spatial detail that would

otherwise be lost during downsampling. The residual connections inside each block ease gradient flow during backpropagation and allow the network to learn additive corrections to an identity mapping, which is particularly appropriate here: the FBP input is already a reasonable (if artifact-corrupted) image, and the network only needs to learn a *residual correction* rather than a full synthesis.

**Generalized multi-view training.** A single ResUNet is trained across all four sparse-view configurations. During each training epoch, the dataloader cycles over all patients and, for each patient, iterates over all four angle keys  $\{180, 90, 60, 45\}$ . Each clean slice is therefore paired with four different FBP inputs corresponding to different levels of angular undersampling. This strategy produces a *generalized* artifact-reduction model that is exposed to the full spectrum of undersampling artifacts during training and can, in principle, perform well across all view counts without retraining.

### 2.3 DiffPIR-Based Diffusion Reconstruction

**Diffusion models as image priors.** The third selected method is a generative approach based on denoising diffusion probabilistic models (DDPMs). A diffusion model defines a probability distribution over clean images through two coupled Markov chains. The *forward* (noising) process gradually corrupts a clean image  $x_0$  towards pure Gaussian noise:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (9)$$

where  $t \in \{1, \dots, T\}$  indexes the diffusion step and  $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$  is the cumulative product of the noise schedule  $\{\beta_s\}$ . At  $t = T$ ,  $\bar{\alpha}_T \approx 0$  and  $x_T$  is approximately standard Gaussian noise. A neural network  $\epsilon_\varphi$  is trained to predict the noise component  $\epsilon$  from the noisy image  $x_t$  and the time step  $t$ ; equivalently, it predicts the *score*  $\nabla_{x_t} \log p(x_t)$  of the data distribution.

**DiffPIR: diffusion for plug-and-play inversion.** DiffPIR [4] adapts the reverse diffusion sampling loop for solving inverse problems. At each reverse step  $t$ , after the standard denoising update, a *data-consistency projection* is applied to enforce agreement with the measured CT sinogram. The data-fidelity term is

$$\mathcal{D}(x) = \frac{1}{2} \|K_\theta x - y^\delta\|_2^2, \quad (10)$$

and its gradient with respect to  $x$  is  $\nabla_x \mathcal{D}(x) = K_\theta^\top (K_\theta x - y^\delta)$ . The data-consistency update is a gradient step of the form

$$x \leftarrow x - \zeta K_\theta^\top (K_\theta x - y^\delta), \quad (11)$$

where  $\zeta > 0$  is a step-size parameter that controls the strength of the data correction. This step is interleaved with the learned reverse-diffusion denoising step at each time  $t$ . Intuitively, the diffusion prior explores the manifold of natural-looking CT images, while the data-consistency step continuously steers the trajectory toward images that are compatible with the measured sparse-view sinogram.

**Trade-off between prior and data.** The parameter  $\zeta$  governs a fundamental trade-off. If  $\zeta$  is too small, the generative prior dominates: the output may be anatomically plausible and visually sharp, but it may hallucinate structures not present in the measured data (a critical failure mode in medical imaging). If  $\zeta$  is too large, the data-consistency correction overwhelms the prior and the reconstruction may reintroduce the original sparse-view streak artifacts, effectively reverting to an FBP-like result. The number of reverse diffusion steps  $T$  and the noise schedule are additional degrees of freedom: fewer steps accelerate inference but may degrade image quality; the schedule determines how fast the model transitions from the learned distribution to the noisy initial state.

### 3 Implementation

#### 3.1 Implementation of Total $p$ -Variation

The TpV pipeline is implemented in `notebooks/01_TpV_reconstruction.ipynb`. The notebook loads the processed Mayo patient file for the test split and selects the fixed central slice across all view settings for consistency. For each view configuration, a CT projector is instantiated as

$$K_\theta = \text{CTProjector}(\text{img\_shape} = (256, 256), \quad \text{angles} = \text{linspace}(0, \pi, n_\theta), \text{det\_size} = 256, \text{geometry} = \text{parallel}). \quad (12)$$

The corresponding TpV solver is then created through `IPPy.solvers.ChambollePockTpVUnconstrained`. The final hyperparameters selected for the test evaluation are

$$\lambda = 0.01, \quad p = 0.35, \quad \text{maxiter} = 150, \quad \text{tolf} = \text{tolx} = 10^{-5}. \quad (13)$$

The solver is initialized from a zero image (`starting_point=None`). The ground-truth clean slice is provided only for diagnostic computation of PSNR and SSIM at each iteration; it does not influence the optimization. The non-negativity constraint  $x \geq 0$  is enforced internally by the solver.

The notebook saves one visual panel per view configuration: each panel contains the ground-truth image, the TpV reconstruction (normalized to  $[0, 1]$ ), and the absolute error map  $|x_{\text{TpV}} - x|$ . Four convergence plots are also saved, containing the objective value, the primal-dual residual, the PSNR curve, and the SSIM curve as functions of iteration number.

#### 3.2 Implementation of Generalized ResUNet

The learned method is implemented in `notebooks/02_ResUnet_reconstruction.ipynb`. The FBP proxy is computed in memory for each sinogram batch using `IPPy.solvers.FBP` applied to the saved noisy sinograms. The resulting FBP images are normalized per image and treated as the degraded input to the network; they are not used as a standalone reconstruction method.

The network is instantiated through `IPPy.nn.models.UNet` with the following configuration:

$$\begin{aligned} \text{ch\_in} &= 1, & \text{ch\_out} &= 1, \\ \text{middle\_ch} &= (32, 64, 128, 256, 512), \\ \text{n\_layers\_per\_block} &= 2, \\ \text{final\_activation} &= \text{None}. \end{aligned} \quad (14)$$

The final layer produces unconstrained logits; outputs are clamped to  $[0, 1]$  only at evaluation time and for display. This choice avoids the vanishing gradient issue associated with a sigmoid activation during training.

Training is supervised with the MSE loss. For each patient batch, the dataloader cycles over all four angle configurations  $\{180, 90, 60, 45\}$ , pairing each sinogram with the corresponding clean slice. The training hyperparameters are

$$\text{learning\_rate} = 10^{-4}, \quad \text{batch\_size} = 8, \quad \text{epochs} = 2, \quad \text{patience} = 3. \quad (15)$$

The optimizer is Adam. The checkpoint `weights/unet/resunet_generalized_latest.pt` stores the model state, optimizer state, loss history, validation loss history, and configuration.

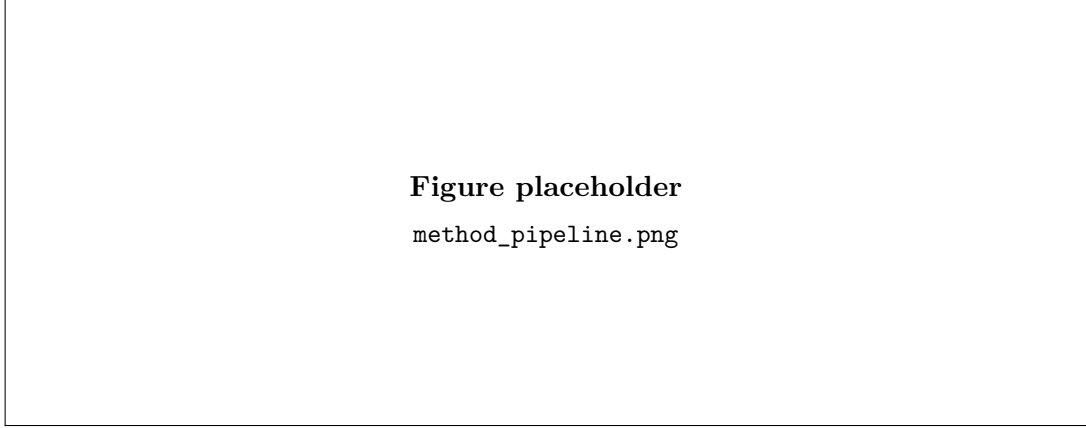


Figure 2: Complete experimental pipeline: from a Mayo CT slice to the sparse-view sinogram, then to TpV, ResUNet, and DiffPIR reconstructions.

### 3.3 Implementation of DiffPIR

The repository configuration selects DiffPIR as the required generative method for the two-student group. No complete DiffPIR reconstruction notebook is currently present in the repository; this subsection defines the theoretical slot that must be completed once the implementation is finalized.

The final DiffPIR implementation must use the same processed Mayo tensors and the same IPPy CT projector as the other methods. For each view count, the algorithm should:

1. Load the fixed noisy sinogram  $y^\delta$  for the shared central test slice.
2. Instantiate `IPPy.operators.CTProjector` to provide  $K_\theta$  and  $K_\theta^\top$ .
3. Initialize the reverse diffusion at  $x_T \sim \mathcal{N}(0, I)$ .
4. At each reverse step  $t = T, T-1, \dots, 1$ : (a) apply the learned denoising step using  $\epsilon_\varphi$ ; (b) apply the data-consistency gradient step (11) with weight  $\zeta$ .
5. Output  $x_0$  as the final reconstruction.

The parameters that must be reported are: the pre-trained denoiser  $\epsilon_\varphi$  used, the total number of reverse diffusion steps  $T$ , the noise schedule  $\{\beta_t\}$ , and the data-consistency weight  $\zeta$ .

## 4 Results and Discussion

### 4.1 Hyperparameter Tuning

**TpV.** Three parameters govern the TpV reconstruction: the regularization weight  $\lambda$ , the gradient-sparsity exponent  $p$ , and the maximum number of Chambolle–Pock iterations. The parameter  $\lambda$  encodes the relative confidence in the prior versus the data. A value of  $\lambda$  that is too small may leave streak artifacts and measurement noise; a value that is too large may over-smooth the image, erasing clinically relevant edges and introducing piecewise-constant stair-casing artifacts (a well-known artifact of TV-type regularization). The exponent  $p$  controls non-convexity: values closer to 0 enforce more aggressive gradient sparsity but make the landscape increasingly non-convex and the optimization less reliable; values closer to 1 approach standard convex TV. The selected value  $p = 0.35$  lies inside the required range  $(0.1, 0.5)$  and provides a balance between sparsity and numerical stability. The number of iterations is set to 150, with early-stopping tolerance `tolf` = `tolx` =  $10^{-5}$ .

**ResUNet.** The key hyperparameters are the learning rate, batch size, and architecture configuration. A learning rate of  $10^{-4}$  is used with the Adam optimizer; the batch size is 8. The architecture uses 5 encoder-decoder levels with base channel count 32 (doubling at each level, reaching 512 at the bottleneck), and 2 residual layers per block. Training proceeds for up to 2 epochs with a patience of 3 epochs for early stopping on validation MSE. These values reflect a preliminary configuration that can be extended as computational resources allow.

**DiffPIR.** The tuning process must be completed after implementation. The key parameter is the data-consistency weight  $\zeta$  in the gradient step (11). Additional parameters include the number of reverse diffusion steps  $T$  (typically 100–1000) and the noise schedule. The expected trade-off is between generative image quality (controlled by the denoising steps) and physical consistency with the measured CT sinogram (controlled by  $\zeta$ ).

## 4.2 Evaluation Metrics

Quantitative evaluation uses PSNR and SSIM, computed on images normalized to  $[0, 1]$ . The PSNR is defined as

$$\text{PSNR}(\hat{x}, x) = -20 \log_{10} \left( \sqrt{\text{MSE}(\hat{x}, x)} \right), \quad \text{MSE}(\hat{x}, x) = \frac{1}{N} \sum_{i=1}^N (\hat{x}_i - x_i)^2, \quad (16)$$

where the images are normalized so that the maximum intensity is 1. Higher PSNR (measured in decibels, dB) indicates lower mean squared error. The formula is equivalent to  $10 \log_{10}(1/\text{MSE})$  for unit-range images.

The SSIM [7] is a perceptual metric that combines luminance, contrast, and structural similarity:

$$\text{SSIM}(\hat{x}, x) = \frac{(2\mu_{\hat{x}}\mu_x + c_1)(2\sigma_{\hat{x}x} + c_2)}{(\mu_{\hat{x}}^2 + \mu_x^2 + c_1)(\sigma_{\hat{x}}^2 + \sigma_x^2 + c_2)}, \quad (17)$$

where  $\mu$ ,  $\sigma^2$  denote local means and variances,  $\sigma_{\hat{x}x}$  is the local cross-covariance, and  $c_1, c_2$  are small stabilization constants. SSIM takes values in  $[-1, 1]$ , with 1 indicating a perfect match. Unlike PSNR, SSIM is sensitive to local structural patterns and is therefore a better predictor of perceived reconstruction quality in medical imaging.

Both metrics are computed on the shared central test slice to enable direct comparison across methods and view counts.

## 4.3 Quantitative Results

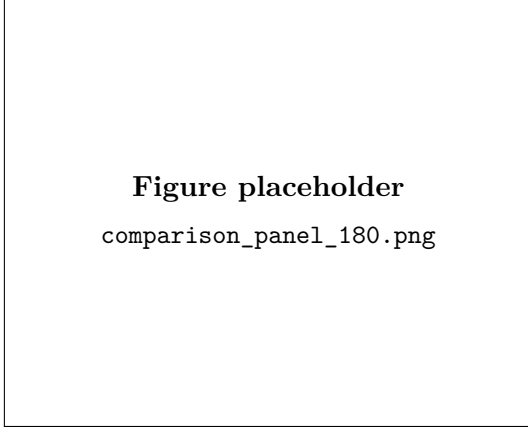
Table 2 collects the PSNR and SSIM values for all methods and view configurations. The ResUNet entries reflect the preliminary scores printed by `notebooks/02_ResUnet_reconstruction.ipynb` on the central test slice after only 2 training epochs. The identical values across all view counts (PSNR 13.84 dB, SSIM 0.1259) indicate that the current checkpoint is undertrained and has not yet converged to a configuration that exploits view-specific artifact information; additional epochs are expected to improve performance and differentiate results across view counts. TpV values must be filled from `notebooks/01_TpV_reconstruction.ipynb`, and DiffPIR values after the third method is implemented.

## 4.4 Visual Results and Critical Discussion

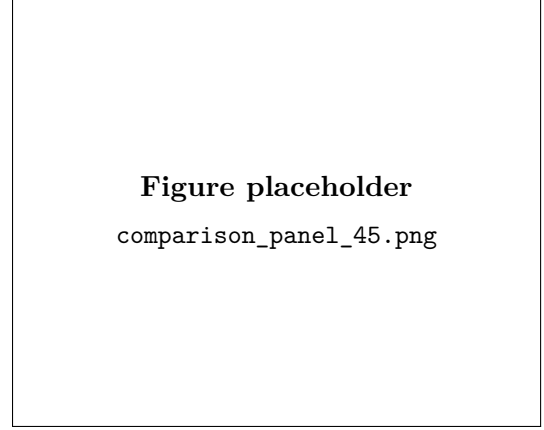
The visual comparison is organized as follows. For each method and each view count, a panel shows (i) the ground-truth slice, (ii) the reconstruction, and (iii) the absolute error map  $|\hat{x} - x|$ . Convergence curves for TpV (objective, residual, PSNR, SSIM vs. iteration) are shown separately. A global comparison panel summarizes PSNR and SSIM across methods and view counts.

Table 2: PSNR and SSIM comparison on the shared Mayo test slice.

| Views | Method              | PSNR (dB)                              | SSIM              |
|-------|---------------------|----------------------------------------|-------------------|
| 180   | TpV                 | <b>TODO: fill</b>                      | <b>TODO: fill</b> |
| 180   | Generalized ResUNet | 13.84                                  | 0.1259            |
| 180   | DiffPIR             | <b>TODO: fill after implementation</b> | <b>TODO: fill</b> |
| 90    | TpV                 | <b>TODO: fill</b>                      | <b>TODO: fill</b> |
| 90    | Generalized ResUNet | 13.84                                  | 0.1259            |
| 90    | DiffPIR             | <b>TODO: fill after implementation</b> | <b>TODO: fill</b> |
| 60    | TpV                 | <b>TODO: fill</b>                      | <b>TODO: fill</b> |
| 60    | Generalized ResUNet | 13.84                                  | 0.1259            |
| 60    | DiffPIR             | <b>TODO: fill after implementation</b> | <b>TODO: fill</b> |
| 45    | TpV                 | <b>TODO: fill</b>                      | <b>TODO: fill</b> |
| 45    | Generalized ResUNet | 13.84                                  | 0.1259            |
| 45    | DiffPIR             | <b>TODO: fill after implementation</b> | <b>TODO: fill</b> |



(a) 180 views



(b) 45 views

Figure 3: Representative comparison panels for high-view and low-view sparse CT settings. Each panel shows the ground truth, TpV reconstruction, ResUNet reconstruction, and their respective absolute error maps.

The final critical discussion must address the following points once all results are available.

**Effect of view reduction.** It should be verified that reconstruction quality (PSNR and SSIM) decreases monotonically as  $n_\theta$  decreases from 180 to 45. The FBP proxy, which is used as the ResUNet input and provides a direct measure of measurement quality, is expected to degrade significantly at 45 views.

**TpV analysis.** TpV should be evaluated in terms of: (i) edge preservation, since the non-convex gradient penalty favors sharp boundaries; (ii) noise suppression, since the data-fidelity term limits amplification of measurement noise; (iii) staircasing artifacts, a known pathology of TV-type regularization where smooth intensity ramps are approximated by staircase functions; and (iv) sensitivity to  $\lambda$  and  $p$ . In particular, the improvement of TpV over FBP is expected to grow as  $n_\theta$  decreases, because the regularization prior becomes more beneficial when the data alone provide less information.

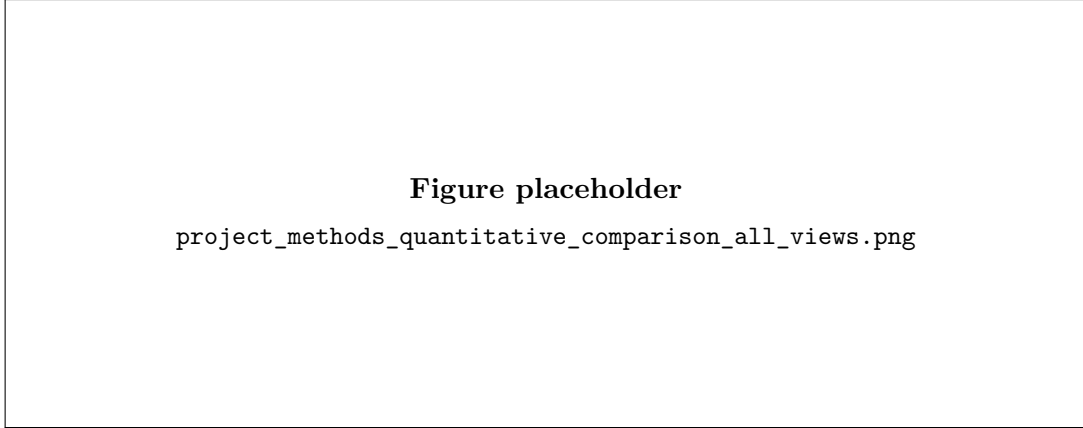


Figure 4: Aggregate PSNR and SSIM across view counts and methods.

**ResUNet analysis.** The ResUNet should be discussed in terms of: (i) artifact removal, assessing whether the network successfully eliminates streak artifacts; (ii) possible over-smoothing, a known consequence of MSE-loss training; (iii) generalization across view counts, verifying whether a single model trained on all four configurations achieves competitive performance at each; and (iv) dependence on the FBP proxy quality, since the input to the network degrades as  $n_\theta$  decreases. The identical PSNR and SSIM values across all view settings in the current checkpoint (Table 2) suggest that the model has not yet learned to differentiate artifact patterns and requires additional training epochs to converge.

**DiffPIR analysis.** Once implemented, DiffPIR should be evaluated by assessing whether the generative prior improves visual quality relative to TpV and ResUNet without violating CT data consistency. A qualitative check should verify that the reconstruction does not hallucinate anatomical structures that are absent from the measured sinogram. The DiffPIR result is expected to be particularly beneficial at the lowest view count (45 views), where the FBP proxy is most severely corrupted and the generative prior has the most opportunity to fill in missing information.

## 5 Conclusions

This report formulates sparse-view CT reconstruction on the Mayo dataset as an ill-posed linear inverse problem and presents a structured comparison of three methodological families. Each family represents a distinct philosophy for incorporating prior knowledge about the unknown image.

The TpV method provides an interpretable model-based strategy: the prior is an explicit non-convex functional on the image gradient, encoding the assumption that CT anatomical images have sparse gradient support. The Chambolle–Pock primal-dual solver handles the resulting non-convex optimization via majorization-minimization with adaptive weights. This approach requires no training data and its behavior is fully determined by the choice of  $\lambda$  and  $p$ ; however, it requires per-image iterative optimization and its expressiveness is limited to the gradient-sparsity assumption.

The Generalized ResUNet provides a supervised end-to-end strategy: the prior is implicit and distributed across the millions of trainable parameters of the network. By training on pairs of FBP inputs and clean targets across all four view configurations, the network learns a data-driven correction function that is fast to evaluate at test time. Its limitation is dependence on the distribution of training data: degradations unseen during training may not be corrected effectively.

The DiffPIR method embeds the CT data-consistency constraint into the reverse trajectory of a score-based diffusion model, combining a powerful generative prior with explicit physics-based guidance. Its implementation and quantitative results remain to be completed.

The final version of the report should insert the missing PSNR and SSIM values, add the generated figures, and replace the DiffPIR placeholders with exact implementation details and experimental results. Once complete, the discussion will compare the three approaches in terms of numerical accuracy, visual artifact characteristics, computational cost, and robustness across sparse-view configurations.

## References

- [1] Cynthia H. McCollough. Tu-fg-207a-04: Overview of the low dose ct grand challenge. *Medical Physics*, 43(6):3759–3760, 2016.
- [2] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4):259–268, 1992.
- [3] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015.
- [4] Yuanzhi Zhu, Kai Zhang, Jingyun Liang, Jiezhong Cao, Bihan Wen, Radu Timofte, and Luc Van Gool. Denoising diffusion models for plug-and-play image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1219–1229, 2023.
- [5] Wim van Aarle, Willem Jan Palenstijn, Jan De Beenhouwer, Thomas Altantzis, Sara Bals, K. Joost Batenburg, and Jan Sijbers. Fast and flexible x-ray tomography using the astra toolbox. *Optics Express*, 24(22):25129–25147, 2016.
- [6] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.
- [7] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.